

Dicionários e conjuntos: organizar e cruzar dados

Questão 1 · Consultar e atualizar por chave

Regista o estado final. Explica a diferença entre atualizar uma chave existente, adicionar uma chave e remover com pop.

Original: 16; Não existe; {'Ana': 16, 'Rui': 14, 'Eva': 18}; True False. Após acrescentar Leo e remover Rui: {'Ana': 16, 'Eva': 18, 'Leo': 10}; o valor devolvido por pop é 14.

```
notas = {'Ana': 16, 'Rui': 12}
notas['Rui'] = 14
notas['Eva'] = 18
notas['Leo'] = 10
removida = notas.pop('Rui')
print(removida)
print(notas)
```

Uma atribuição a chave existente substitui o valor; a uma chave nova acrescenta um par. pop remove o par e devolve o valor associado.

Questão 2 · Contar ocorrências

Regista as contagens antes e depois de normalizar. Explica por que Ana e ana eram chaves diferentes.

Original: ana 3, rui 2, eva 1. Com 'Ana ana ANA Rui', sem normalizar: Ana 1, ana 1, ANA 1, Rui 1. Após lower: ana 3, rui 1.

```
texto = 'Ana ana ANA Rui'
frequencias = {}
for palavra in texto.lower().split():
    frequencias[palavra] = frequencias.get(palavra, 0) + 1
for palavra, quantidade in frequencias.items():
    print(palavra, quantidade)
```

Strings com maiúsculas diferentes são chaves distintas. Normalizar antes da contagem permite tratá-las como a mesma palavra.

Dicionários e conjuntos: organizar e cruzar dados

Questão 3 · Cruzar inscrições

Que conjuntos mudaram e porquê? Explica por que adicionar um elemento já existente não aumenta o tamanho.

Original: ambas [102]; só Python [101, 103]; pelo menos uma [101, 102, 103, 104]. Após adicionar 104: ambas [102, 104]; só Python [101, 103]; união mantém os quatro números. `len(python)` é 4.

Adicionar 102 novamente não muda o conjunto nem o seu tamanho. Conjuntos guardam elementos únicos. `sorted` apenas organiza a apresentação; um set não garante ordem de apresentação.

Questão 4 · Guardar dados para outra execução

Compara os dados antes de guardar e após recuperar. Explica os modos `wb` e `rb` e por que a origem do ficheiro é relevante.

Após a alteração, alunos contém 101: Ana, 102: Rui, 103: Eva; inscritos contém 101 e 103. Os dados recuperados mantêm os mesmos pares e elementos. A ordem do set não é relevante.

`wb` abre em escrita binária e `rb` em leitura binária. `pickle` serializa objetos em bytes e recupera-os. A leitura de `pickle` pode executar código, pelo que só se devem usar dados de origem confiável, como o ficheiro criado na própria atividade.

```
# Acrescentar antes do bloco que guarda:  
dados['alunos'][103] = 'Eva'  
dados['inscritos'].add(103)
```

Dicionários e conjuntos: organizar e cruzar dados

Questão 5 · Desafio final - uma solução possível

```
def inscrever(alunos, inscritos, numero):  
    if numero not in alunos or numero in inscritos:  
        return False  
    inscritos.add(numero)  
    return True  
  
alunos = {101: 'Ana', 102: 'Rui', 103: 'Eva'}  
inscritos = {101}  
print(inscrever(alunos, inscritos, 102))  
print(inscrever(alunos, inscritos, 102))  
print(inscrever(alunos, inscritos, 999))  
for numero in sorted(inscritos):  
    print(numero, alunos[numero])
```

As chamadas devolvem True, False e False. Saída dos inscritos: 101 Ana e 102 Rui. O dicionário mantém os três alunos; o conjunto final é {101, 102}. O teste de pertença ao dicionário impede inscrições de números desconhecidos.